

PROJECT REPORT

Automated Analysis of Participant Feedback Using Natural Language Processing

Author: Anastasiia Skryzhadlovskia

Professor: Fabio Tamburini

July 29, 2026

Contents

1	Introduction	3
2	Project Objectives	3
3	Dataset	3
3.1	Synthetic dataset	3
3.2	Real-world validation datasets	4
3.3	Data Ethics	4
4	System Architecture	4
5	Implementation Details	4
5.1	Technology Stack	5
5.2	Main Pipeline	5
5.3	Project Structure	6
5.4	Preprocessing Module	6
5.5	Sentiment Analysis Module	7
5.6	Learning Detection Module	7
5.7	Embedding Module	7
5.8	Taxonomy and Learning Outcome Classification	7
5.9	Clustering Module	8
5.10	Quote Extraction Module	8
6	Results	8
6.1	Example Output	9
6.2	Manual Evaluation	10
6.2.1	Methodology	10
6.2.2	Error Analysis	10
7	Discussion	11
8	Future Work	12
9	Conclusion	12

1 Introduction

International youth mobility projects, training courses, and exchanges usually collect feedback from participants after each activity. This feedback is valuable because it helps organizers understand what worked well, what could be improved, and what participants learned during the project. However, manually reading all responses and preparing evaluation reports can be very time-consuming and inefficient.

The goal of this project was to develop a Natural Language Processing (NLP) pipeline that automatically analyzes participant feedback and generates a structured summary that is essential for project evaluation and report preparation. The system identifies positive and negative comments, detects learning outcomes of participants, groups similar comments together, and extracts representative quotes to better understand the feedback summary.

The project was inspired by real Erasmus+ youth mobility projects, where organizers always need to write an evaluation section in the final project report, usually by summarizing participant experiences and learning outcomes. The developed tool aims to reduce manual effort while preserving important information from participant responses.

2 Project Objectives

The main objective of this project was to create a system capable of automatically processing participant feedback and generating useful summaries.

The specific objectives were:

- Load and preprocess feedback stored in CSV files.
- Split responses into individual sentences.
- Classify sentences according to sentiment.
- Detect learning-related statements.
- Assign learning statements to Erasmus+ learning outcome categories and project-specific topic labels.
- Group similar comments using clustering.
- Extract representative quotes.
- Generate structured output suitable for evaluation reports.

3 Dataset

3.1 Synthetic dataset

A synthetic dataset (`feedback_test_dataset.csv`) was created for initial development and testing. It contained approximately 60 participant responses representing feedback collected after youth exchanges and training activities. The dataset included positive comments, negative comments, learning-related statements, and ambiguous sentences designed to challenge sentiment analysis.

Examples of positive comments from the generated dataset:

“The trainers were incredibly supportive and helpful.”

Examples of negative comments:

“The schedule was too intense and there were not enough breaks.”

Examples of learning statements:

“I became more confident speaking English.”

During development, the synthetic dataset was gradually made more complex by adding complex and ambiguous sentences, as well as inconsistent punctuation and emojis, which are often part of real feedback forms.

3.2 Real-world validation datasets

After the pipeline was validated on synthetic data, it was applied to two real Erasmus+ feedback datasets:

- `project1_real_data.csv` — 43 responses from a sustainability training course (`project_id = Nature_Training_Course`), producing 58 sentences after segmentation.
- `project2_real_data.csv` — 65 responses from 39 participants in a career-focused youth exchange (`project_id = Career_Youth_Exchange`), producing 79 sentences.

Together, the two real datasets contain **137 sentences** used for manual evaluation.

All input files used the same CSV structure with four required columns: `project_id`, `response_id`, `question`, and `text`. The `project_id` value selects the project-specific learning topic used during outcome matching. Real files exported from Excel used semicolon separators, which the preprocessing module supports automatically.

3.3 Data Ethics

The real datasets consist of participant feedback collected from Erasmus+ projects I conducted. Responses were exported from Google Forms as CSV files without participant names or other information. Only `response_id` values were retained to link multiple sentences from the same answer. The data was used solely for developing and evaluating the NLP pipeline within this academic project.

4 System Architecture

The system consists of multiple processing stages arranged in a pipeline.

Feedback CSV → Preprocessing → Sentiment Analysis → Learning Detection → Outcome Classification → Clustering → Quote Extraction → Final JSON Report

Each stage performs a specific task and passes its output to the next stage. The full pipeline is implemented in `src/pipeline.py` and can be executed from the command line (`run_evaluator_all_steps.py`) or through a Streamlit web interface (`app.py`).

5 Implementation Details

The project was implemented in Python using a modular architecture. Each module is responsible for a specific task.

5.1 Technology Stack

The pipeline relies on the following libraries and tools:

- **Python 3** — core programming language
- **pandas** — loading and processing CSV feedback data
- **spaCy** (`en_core_web_sm`) — sentence segmentation and text preprocessing [2]
- **NLTK / VADER** — baseline sentiment analysis [1]
- **sentence-transformers** (`all-MiniLM-L6-v2`) — semantic embeddings for outcome matching and clustering [3]
- **scikit-learn** — K-Means clustering [4]
- **Streamlit** — optional web interface for running the pipeline

All dependencies are listed in `requirements.txt`.

The pipeline can also be run through a Streamlit web interface (`app.py`), which allows organizers to select a CSV file, run the analysis, and view strengths, improvements, and learning outcomes in the browser. Figure 1 shows the interface after processing a real dataset.

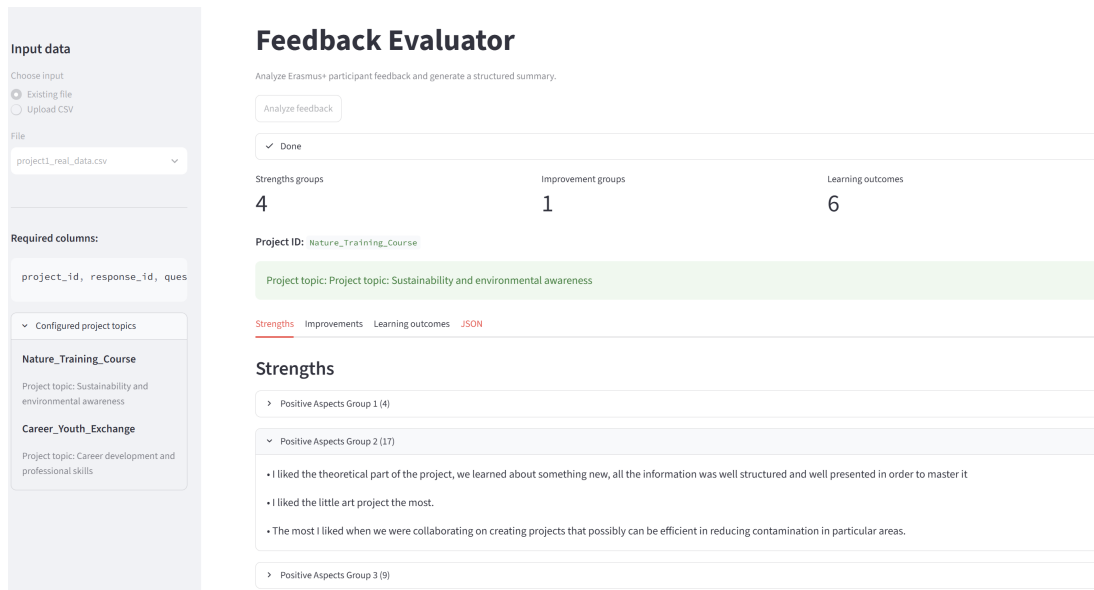


Figure 1: Streamlit interface showing pipeline results for a real feedback dataset

5.2 Main Pipeline

The pipeline logic is centralized in `src/pipeline.py`. The entry points are:

- `run_evaluator_all_steps.py` — command-line execution
- `app.py` — optional Streamlit UI for uploading CSV files and viewing results, which was created with the help of AI in order to have some visual interface for the project.

The pipeline executes the following stages:

1. Load feedback data

2. Preprocess text and split into sentences
3. Perform question-aware sentiment analysis
4. Detect learning-related sentences
5. Classify learning outcomes (Erasmus+, project topic, or Other)
6. Cluster positive and negative comments separately
7. Extract representative quotations
8. Generate structured JSON output

5.3 Project Structure

The codebase is organised in the following way: Table 1 summarises the main folders and files.

Table 1: Organisation of the FeedbackEvaluator project

Location	Role
Project root	<code>run_evaluator_all_steps.py</code> (CLI), <code>app.py</code> (Streamlit UI), <code>requirements.txt</code>
<code>src/</code>	Core modules: <code>pipeline.py</code> , <code>preprocess.py</code> , <code>sentiment.py</code> , <code>learning_filter.py</code> , <code>embeddings.py</code> , <code>learningoutcomes_detector.py</code> , <code>taxonomy.py</code> , <code>clustering.py</code> , <code>quotes.py</code>
<code>data/raw/</code>	Input CSV feedback files
<code>data/processed/</code>	Intermediate pipeline outputs
<code>evaluation/</code>	Manual labels and metrics scripts
<code>outputs/</code>	Final JSON reports

5.4 Preprocessing Module

The file `preprocess.py` is responsible for loading and preparing feedback data.

The module:

- Loads CSV files
- Cleans text
- Splits responses into individual sentences

Sentence segmentation is performed using spaCy [2]. This allows the system to analyze individual statements rather than entire responses. For example:

"The trainers were helpful. The schedule was too intense."

This sentence contains both positive and negative feedback. Sentence-level analysis allows these opinions to be separated correctly.

5.5 Sentiment Analysis Module

The file `sentiment.py` performs sentiment classification using the VADER sentiment analyzer [1], extended with custom rules and **question-aware logic**.

Each sentence receives:

- A sentiment label (**positive**, **negative**, or **neutral**)
- A sentiment score

The function `get_sentiment(text, question)` uses the survey question as additional context. For example, on improvement questions (“What could be improved?”), short answers such as “no” or “nothing to add” are treated as positive, because they indicate that the participant has no criticism. Similarly, phrases like “no negative aspects” are classified as positive rather than negative.

During testing, several problems were observed with VADER alone. For example:

“The accommodation could have been better organized.”

Although this sentence expresses criticism, VADER sometimes classified it as positive.

To improve performance, custom rules were introduced for phrases such as “could have been better”, “should have been better”, “not enough”, “too intense”, and similar patterns. Rule-based checks are applied before the VADER score. After these adjustments, the approach produced more accurate results on real participant feedback.

5.6 Learning Detection Module

The file `learning_filter.py` identifies sentences that describe learning or personal development using keyword cues such as **learned**, **gained**, **improved**, **developed**, and **became**.

In addition, the pipeline marks a sentence as learning-related if the survey question itself asks about learning (for example, “What did you learn?”), even when the answer does not contain an explicit learning verb.

Examples include:

“I improved my communication skills.”

“I became more confident speaking English.”

5.7 Embedding Module

The file `embeddings.py` loads the Sentence Transformer model [3]:

```
all-MiniLM-L6-v2
```

The model converts sentences into numerical vectors, these embeddings are later used for learning outcome classification and clustering, described below.

5.8 Taxonomy and Learning Outcome Classification

Outcome categories are defined in `taxonomy.py`. The system uses three types of labels:

1. **10 official Erasmus+ outcomes** [5], including improved learning performance, enhanced employability, increased self-empowerment, improved language and digital competences, enhanced intercultural awareness, and others.

2. **One project-specific topic** per `project_id`, defined in `PROJECT_TOPICS`. For the real datasets, these are “Sustainability and environmental awareness” and “Career development and professional skills”, both were project topics, data from which was used. It was decided to include this category during the later stage of development, since often it is asked to include project-specific outcomes in the report as well, rather than general Erasmus+ outcomes.
3. **Other** — assigned when no category reaches the similarity threshold. It was also useful to identify such sentences for further inspection, because often they are vague or have some grammar/vocabulary issues.

The file `learningoutcomes_detector.py` implements the `OutcomeMatcher` class. The process consists of four steps:

1. Generate sentence embeddings
2. Generate category embeddings from descriptions and example sentences
3. Calculate cosine similarity between the sentence and each category
4. Select the best matching category if the score is at least **0.40**; otherwise assign **Other**

Cosine similarity is calculated between sentence embeddings and category embeddings:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

The category with the highest similarity score is selected. This design allows the system to capture both generic Erasmus competences and project-specific learning (for example, sustainability or career skills) within the same output structure.

5.9 Clustering Module

The file `clustering.py` groups semantically similar comments using K-Means clustering [4]. Positive and negative comments are clustered separately. The number of clusters is chosen dynamically based on the amount of available feedback (`choose_n_clusters`), rather than using a fixed value for every project.

K-Means assigns numeric cluster IDs (0, 1, 2, ...). In the JSON output, these appear as generic labels such as “Positive Aspects Group 1” until they are mapped to human-readable theme names in `outputs/cluster_labels.json`. The author inspects representative quotes for each cluster and updates this file with descriptive labels (for example, “Engaging activities” or “Schedule concerns”) before the final report is shared with organizers. In the current implementation, this step is manual rather than automatic.

5.10 Quote Extraction Module

The file `quotes.py` extracts representative quotes.

The algorithm identifies the sentence closest to the cluster centroid and selects it as the most representative quote. These quotations can be directly included in evaluation reports, which are often encouraged since they help readers understand what participants think about the project without any paraphrasing.

6 Results

The pipeline successfully processed participant feedback and generated structured JSON outputs for both real datasets.

6.1 Example Output

Listing 1 shows an excerpt from the generated report for the career-focused youth exchange (project2_real_data). The full file is saved as outputs/project2_real_data_final_output.json.

Listing 1: Excerpt from pipeline JSON output (project2)

```
{
  "project_id": "Career_Youth_Exchange",
  "project_topic_category":
    "Project topic: Career development and professional skills",
  "strengths": [
    {
      "cluster_id": 1,
      "count": 15,
      "representative_quotes": [
        "Activities where we had to talk about personal
        experiences, learning from others and having to talk
        about what success means for each of us.",
        "I was happy that we were introduced to some activities
        like coming up with our own ted talks, ikigai, and
        the river session."
      ]
    }
  ],
  "improvements": [
    {
      "cluster_id": 0,
      "count": 5,
      "representative_quotes": [
        "Also, it wasn't 100 percent clear where the location
        was and this could be improved further."
      ]
    }
  ],
  "learning_outcomes": [
    {
      "label": "Enhanced positive interactions with people
        from different backgrounds",
      "count": 4,
      "representative_quotes": [
        "I improved my empathy and kindness as well as my
        self-confidence.",
        "I became more open and communicative."
      ]
    }
  ],
  {
    "label": "Project topic: Career development and
      professional skills",
    "count": 4,
    "representative_quotes": [
      "Teamwork, public speaking and time management",
      "Yes, my biggest improvement would be public speaking,
      but besides i also developed my constructive thinking
      and team work."
    ]
  }
]
```

The final output is generated as a JSON file with all the categories, and supported by the representative quotes.

This structure allows organizers to copy strengths, improvement areas, and learning outcomes, together with participant quotes, directly into a project evaluation report, making the process much faster.

6.2 Manual Evaluation

6.2.1 Methodology

All 137 sentences from both real datasets were manually labeled by me in `evaluation/eval_full.csv`, in order to check if the pipeline is performing well. Labeling was performed in Excel, where I assigned sentiment to each sentence and learning outcome to the sentences that describe learning. The learning outcome labels were taken from `evaluation/outcome_labels.txt`, which lists all 10 Erasmus+ categories, both project topic labels, and `Other`.

Each row was annotated with:

- `gold_sentiment` — manual sentiment label (`positive`, `negative`, or `neutral`)
- `gold_is_learning` — whether the sentence describes learning or personal development
- `gold_outcome` — the correct outcome category (filled only for learning sentences)

All sentences were labeled by me, which is definitely a limitation. However, the feedback evaluation is very subjective in general, and the idea was to create something that could make the process faster and more efficient but not to eliminate the human assessment part completely, which is still crucial here.

Metrics were computed with `evaluation/compute_eval_metrics.py` by comparing gold labels to pipeline predictions. Table 2 summarizes the results.

Table 2: Manual evaluation results on 137 real-dataset sentences

Metric	Accuracy
Sentiment (with question context)	73.7%
Sentiment (baseline, no question)	71.5%
Learning detection	97.1%
Outcome matching (47 learning sentences)	68.1%

Question-aware sentiment improved accuracy by 2.2 percentage points compared to the baseline that ignores the survey question. Learning detection performed strongly (97.1%), while outcome matching remained the most challenging step (68.1%), especially when distinguishing between similar Erasmus categories and project-specific topics. Possible reasons for such results are discussed in the next section.

6.2.2 Error Analysis

Inspecting mismatches between manual labels and pipeline predictions revealed three main error types:

1. **Short or fragment answers.** Single-word or very brief responses such as “Public speaking” were often classified as `Other` because they lack enough semantic context for embedding-based matching, even when the survey question clearly asks about learning.

2. **Erasmus vs. project topic confusion.** The system sometimes assigns a generic Erasmus label when the gold label is the project topic, or vice versa. For example, “Plus some management skills” was predicted as “Enhanced employability and improved career prospects” but manually labeled as “Project topic: Career development and professional skills”. In general, this is a very subjective assessment, where also multiple categories might be suitable.
3. **Similar Erasmus categories.** Closely related outcomes are difficult to separate. For instance, “Especially our human relations and social ties have become much stronger” was predicted as “Enhanced intercultural awareness” but labeled as “Enhanced positive interactions with people from different backgrounds”. This is another challenge

These patterns explain why outcome accuracy (68.1%) lags behind learning detection (97.1%). After manual labelling it was also concluded that even for human evaluators it is very difficult to assign a correct category to each sentence, since multiple categories could be suitable. One solution is to add multi-label assignment feature in future pipeline versions. Luckily, report writing does not demand such detailed analysis of the feedback, rather just general summaries with quotes, so the project still performs well, given this.

7 Discussion

The developed system demonstrates that NLP techniques can support the analysis of participant feedback from youth mobility projects. Rather than replacing human judgment entirely, the pipeline acts as a structured first pass: it separates opinions at sentence level, highlights learning-related statements, and groups similar comments so that organizers can focus on reviewing themes and quotes instead of reading every response from scratch.

One of the main strengths of the project is its modular architecture. Preprocessing, sentiment analysis, learning detection, outcome matching, clustering, and quote extraction are implemented as separate components, which means each stage can be improved independently without rebuilding the entire system. This design also made it possible to introduce question-aware sentiment analysis and project-specific outcome categories as targeted enhancements after initial testing on synthetic data.

The use of official Erasmus+ learning outcomes, combined with project-specific topic labels, makes the results directly relevant for project evaluation. Organizers can report both generic competences (such as intercultural awareness or self-confidence) and theme-specific learning (such as sustainability or career skills) within a single output structure. The manual evaluation on 137 real sentences supports this claim in part: learning detection reached 97.1% accuracy. Outcome matching, at 68.1%, shows that the approach is useful but requires human review.

Several limitations should be acknowledged. The evaluation was conducted on a relatively small sample of 137 sentences drawn from only two projects, so the reported accuracies may not generalise to other activity types. Sentiment and learning detection also depend on manually defined rules and keyword lists, which means new phrasing in participant feedback may not always be handled correctly unless the rules are updated. Clustering produces numbered groups that must be interpreted and renamed manually, since the system identifies similar comments but does not yet generate final theme names on its own. Outcome classification remains weak but still helpful. It is especially challenging for both human evaluators and the pipeline, when statement could reasonably fit multiple categories.

Despite these limitations, the system significantly reduces the amount of manual work required when analyzing participant feedback. For real Erasmus+ projects, where evaluation forms often contain dozens of open-ended answers, even partial automation can save time and help produce a more consistent summary. With further testing on larger datasets and refinements to

outcome matching, the tool could become a practical support instrument for youth workers and project coordinators preparing final reports.

8 Future Work

Several directions could extend this project. Testing on larger and more diverse real-world datasets would provide a clearer picture of how well the pipeline generalises across different project types and survey formats. Automatic cluster naming would remove the need to manually edit labels.

The existing Streamlit interface could be extended with features for reviewing and correcting labels before export. Finally, integrating a large language model for draft report generation could further reduce the time needed to prepare Erasmus+ evaluation sections, while keeping human review as the final step.

9 Conclusion

This project presented an NLP pipeline for automated analysis of participant feedback from youth mobility projects.

The system combines preprocessing, question-aware sentiment analysis, learning outcome detection, semantic similarity matching against Erasmus+ and project-specific categories, dynamic clustering, and quote extraction. A manual evaluation on 137 real sentences confirmed that the pipeline performs well on learning detection and reasonably on sentiment, while outcome classification remains an area for further improvement.

The optional Streamlit interface makes the tool accessible to organizers who may not use the command line directly.

References

- [1] Hutto, C.J. and Gilbert, E.E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. In *Proceedings of the International AAAI Conference on Web and Social Media*, 8(1), 216–225.
- [2] Honnibal, M., Montani, I., Van Landeghem, S., and Boyd, A. (2020). spaCy: Industrial-strength Natural Language Processing in Python. <https://spacy.io>
- [3] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 3982–3992.
- [4] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [5] European Commission (2024). Erasmus+ Programme Guide — Annex I: Glossary and definitions. <https://erasmus-plus.ec.europa.eu/programme-guide>